claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_e3cc36d8-eca\agent-a3d2614df8c82d6db.jsonl`
- SHA-256: `e50c66f92a1d4c8a28508ccb5dbbda49d3f7bb0d283567072c08591649435a59`
- Source modified: `2026-06-21T09:20:39+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_e3cc36d8-eca`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T09:19:21.057Z)

Engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer. Your job: ADVERSARIALLY REFUTE this claim, do not confirm it.

CLAIM: "Under default single-user config, POST /api/process accepts a gs://|gcs:// video_path, fetches+processes it, and writes a video_id that db.get_video() returns and the SPA can load via ?video=."

YOUR LENS: SUBMIT-TIME GATE: focus ONLY on backend/main.py process_video() submit path ? validate_input_path, resolve_input_ref ValueError mapping, the input_ref.backend=="local" existence check, the segmenter check. Find any branch that 400s/404s a valid gs:// URI under default config.

Read the actual code for your lens and try hard to find a CONCRETE counterexample (file:line) where a valid gs:// URI under default config fails to yield a queryable API-DB video_id. IMPORTANT: a failure of the real network fetch due to a MISSING dependency (google-cloud-storage) or MISSING auth (ADC) is ENVIRONMENTAL, not a code gate ? do NOT count those as a refutation; the claim is about code routing/acceptance, assuming the fetch itself can run. Only refute with a real code-path defect. Default refuted=true ONLY if you have a concrete counterexample; otherwise refuted=false. Return structured JSON.

2. assistant (2026-06-21T09:19:24.811Z)

I'll analyze the submit-time gate in `process_video()` to find any branch that rejects a valid `gs://` URI under default config. Let me start by reading the relevant code.

3. user (2026-06-21T09:19:25.426Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       from urllib.parse import quote
7       import uvicorn
8       from dotenv import load_dotenv
9
10      load_dotenv()
11
12      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13      logging.basicConfig(
14          level=logging.INFO,
15          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16          datefmt='%H:%M:%S'
```

```
17      )
18      logger = logging.getLogger(__name__)
19      from fastapi import FastAPI, HTTPException
20      from fastapi.middleware.cors import CORSMiddleware
21      from pydantic import BaseModel
22      from typing import List, Dict, Any, Optional
23
24      from fastapi.staticfiles import StaticFiles
25      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26      from backend.pipeline.validators.base import registry
27      from backend.infrastructure.database import Database
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.stitcher.compiler import VideoCompiler
30      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
31      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
33      from backend.results import Failure  # standardized error/result contract
34      from backend.reporting import emit_indexer_report
35      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
36
37      app = FastAPI(title="Sports Highlight Indexer API")
38
39      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
40      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
41      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
42      app.add_middleware(
43          CORSMiddleware,
44          allow_origins=["*"],
45          allow_credentials=False,
46          allow_methods=["*"],
47          allow_headers=["*"],
48      )
49
50      # Serves static frontend files directly (now under api/ per approved structure)
51      os.makedirs("backend/api/static", exist_ok=True)
52      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
53
54      @app.get("/", response_class=HTMLResponse)
55      def serve_index():
56          index_path = "backend/api/static/index.html"
57          if os.path.exists(index_path):
58              with open(index_path, "r", encoding="utf-8") as f:
59                  return f.read()
60          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
61
62      # Global variables initialized at startup
63      db_instance: Optional[Database] = None
64      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
65
66      # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
67      # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
68      # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
69      # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
70      # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
71      # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
72      # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
73      from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
```

```
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _MAX_DRAIN_WAKE_SEC,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    @app.get("/api/health")
    def health():
        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
        confirm the engine is reachable and which detector is wired. Never raises."""
        sport = getattr(global_config, "sport", None)
        indexing = getattr(global_config, "indexing", None)
        return {
            "status": "ok",
            "sport": sport,
```

```
137             "default_segmenter": getattr(indexing, "default_segmenter", None),
138         }
139
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on
144         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145         the sync endpoint used to return (UI compatibility); the per-video observability
146         report is still emitted in `finally:` and grafted as response["reports"].
147
148         `progress` is an optional callable(stage: str) used to surface coarse job progress.
149         Raises on unexpected internal errors ? the caller records those as a job failure
150         (after this function has already restored the pre-run segments, see below).
151         """
152         notify = progress or (lambda stage: None)
153
154         notify("hashing")
155         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
156         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
157         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
158         # consumers (hash / validators / segmenter) use the resolved local path.
159         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160         video_id = compute_video_id(local_video)
161         was_reingest = db_instance.get_video(video_id) is not None
162
163         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
164         notify("validating")
165         logger.info(f"Running validations for {req.video_path}...")
166         val_results, val_context = registry.run_all_with_context(local_video, global_config)
167         failures = [r for r in val_results if not r.passed]
168
169         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
170         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
171         seg_name = req.segmenter or default_seg
172         segmenter_actual = None
173         segmentation_ran = False
174         response: Optional[Dict[str, Any]] = None
175         try:
176             if failures:
177                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
178                 # status; it no longer destroys the video's prior good segments.
179                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
180                 response = {
181                     "video_id": video_id,
182                     "status": "failed",
183                     "message": "Video failed validation checks.",
184                     "results": [r.model_dump() for r in val_results],
185                 }
186                 return response
187
188             # 2. Run segmenter & indexer
189             logger.info("[API] Video passed checks. Segmenting and indexing...")
190             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
191
192             segmenter_cls = segmenter_registry.get(seg_name)
193             if not segmenter_cls:
```

```
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
```

```
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
```

```
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
```

```
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "
f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
```

```
            # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
            # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
            download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
            return {"status": "success", "filepath": out_path, "download_url": download_url}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for _ in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

```

```
        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess
            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts", "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

```

```
        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
        # the executor is in-process. Mark them failed so pollers see a terminal state.
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
```

```
            print(f"[CLI] Processing video file: {args.video_path}")
            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
            storage_cfg = getattr(global_config, "storage", None)
            try:
                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
            except ValueError as e:
                print(f"[FAIL] {e}")
                return
            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return
            local_video = storage.acquire_local_input(args.video_path, storage_cfg)

            video_id = compute_video_id(local_video)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(local_video,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
```

```
652                     print("[FAIL] Indexing halted for video.")
653                     segments = []
654                 else:
655                     segments = result.value if hasattr(result, "value") else result
656                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
657                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
658                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
659             finally:
660                 # Always emit the per-video observability report. Never raises.
661                 paths = emit_indexer_report(
662                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
663                     val_results=val_results, val_context=val_context,
664                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
665                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
666                 )
667                 if paths:
668                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
669                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
670                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
671             return
672
673         # Start Local Server Mode
674         if args.server or not args.video_path:
675             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
676             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
677             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
678             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
679             _get_executor().submit(_drain_due_jobs)
680             uvicorn.run(app, host="127.0.0.1", port=args.port)
681
682     if __name__ == "__main__":
683         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
684         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
685         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
686         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
687         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
688         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
689         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
690         import backend.main as _bm
691
692         _bm.main()
693
```

4. assistant (2026-06-21T09:19:28.896Z)

The submit-path is in `process_video()` at lines 260-300. I need to trace three things for a `gs://` URI under default config: (1) `validate_input_path`, (2) `resolve_input_ref` ValueError mapping, (3) the `input_ref.backend == "local"` existence check, and the segmenter check. Let me read the storage and paths modules, plus the default config.

5. user (2026-06-21T09:19:29.682Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
```

```
      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are imported
      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and preserves
      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally MOUNTED
      Google Drive (Google Drive for Desktop) as plain files.

      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an identity
      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
      """
      from __future__ import annotations

      import os
      import tempfile
      from typing import Any, Dict, Iterator, Optional

      from backend.storage.base import StorageBackend
      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

      __all__ = [
          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
          "get_storage", "fetch", "put", "list_uris",
          "acquire_local_input", "input_is_local", "resolve_input_ref",
      ]


      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
          if storage_config is None:
              return {}
          if hasattr(storage_config, "model_dump"):
              return storage_config.model_dump()  # type: ignore[no-any-return]
          return dict(storage_config)


      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under the input
          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported URI scheme ?
          callers at the API/CLI boundary catch that and return a clean client error (never a 500).

          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL path (e.g.
          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI string."""
          sc = _storage_dict(storage_config)
          uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
                                  input_root=sc.get("input_root", ""))
          return parse_uri(uri)


      def input_is_local(raw: str, storage_config: Any = None) -> bool:
          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ? i.e. a
          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive URI
          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
          ``resolve_input_ref``)."""
          return resolve_input_ref(raw, storage_config).backend == "local"
```

```
57
58
59      def acquire_local_input(raw: str, storage_config: Any = None, *,
60                              scratch_parent: "str | None" = None) -> str:
61          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66          path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67          regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68          worker's ``cmd_infer`` already ships."""
69          sc = _storage_dict(storage_config)
70          ref = resolve_input_ref(raw, sc)
71          if ref.backend == "local":
72              return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
73          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74          return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77      def _backend_class(name: str):
78          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79          if name == "local":
80              from backend.storage.local import LocalStorage
81              return LocalStorage
82          if name == "s3":
83              from backend.storage.s3 import S3Storage
84              return S3Storage
85          if name == "gcs":
86              from backend.storage.gcs import GCSStorage
87              return GCSStorage
88          if name == "gdrive":
89              from backend.storage.gdrive import GDriveStorage
90              return GDriveStorage
91          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94      def get_storage(backend: Optional[str] = None, *,
95                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97          the implementation; its per-backend settings come from ``config[backend]`` merged
with
98          ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99          cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107               config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
```

```
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127
```

6. user (2026-06-21T09:19:30.099Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Any, Optional
4
5
6       def output_base(config: Any) -> str:
7           """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8           (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
9           chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10          a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12          This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13          ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy
dict-shim
14          (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
15          try:
16              out = config.get("output", {}).get("output_dir")
17          except Exception:
18              out = None
19          return out or "output"
20
21
22      def safe_output_filename(name: str) -> str:
23          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
24
25          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
26          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
27          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
28          """
29          if not name or not name.strip():
30              raise ValueError("output_filename must not be empty")
31          if "\x00" in name:
```

```
32              raise ValueError("output_filename must not contain a null byte")
33          if "/" in name or "\\" in name:
34              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
35          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
36              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
37          return name
38
39
40      def resolve_under_root(path: str, root: str) -> str:
41          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
42          rp = os.path.realpath(path)
43          rr = os.path.realpath(root)
44          try:
45              contained = os.path.commonpath([rp, rr]) == rr
46          except ValueError:  # different drives on Windows
47              contained = False
48          if not contained:
49              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
50          return rp
51
52
53      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
54          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
57          so the local 'point it at any of my videos' workflow is preserved. Setting an
58          allowed_root (a future hosted/tenant deployment) enforces containment.
59          """
60          if "\x00" in path:
61              raise ValueError("video_path must not contain a null byte")
62          if allowed_root:
63              resolve_under_root(path, allowed_root)
64
```

7. assistant (2026-06-21T09:19:34.396Z)

Now let me look at the `parse_uri` and `resolve_input_uri` functions in `ref.py`, since the claim uses `gs://|gcs://`. The backend class map has `gcs` but I need to check how `gs://` maps to a backend name.

8. user (2026-06-21T09:19:34.731Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
```

```
    from datetime import datetime
    from typing import Optional

    _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
    _KNOWN = ("local", "s3", "gcs", "gdrive")


    @dataclass(frozen=True)
    class StorageStat:
        """Lightweight object metadata (uniform across backends)."""
        size: int
        modified: Optional[datetime] = None
        etag: Optional[str] = None
        content_type: Optional[str] = None


    @dataclass(frozen=True)
    class StorageRef:
        """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
        empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
        backend: str
        bucket: str
        key: str
        uri: str

        @property
        def name(self) -> str:
            return self.key.rstrip("/").rsplit("/", 1)[-1]


    def parse_uri(uri: str) -> StorageRef:
        """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.

        A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
        have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
        """
        s = str(uri)
        m = _SCHEME_RE.match(s)
        if not m:
            return StorageRef("local", "", s, f"local://{s}")
        scheme = m.group(1).lower()
        rest = s[m.end():]
        if scheme == "gs":
            scheme = "gcs"
        if scheme in ("local", "gdrive"):
            # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
            return StorageRef(scheme, "", rest, s)
        if scheme in _KNOWN:
            bucket, _, key = rest.partition("/")
            return StorageRef(scheme, bucket, key, s)
        raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")


    def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
        """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).

        Precedence (first match wins):
          1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
```

```
76            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80               ``"gcs://bucket/x"``).
81            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
84          path used today, preserving byte-for-byte behaviour.
85          """
86          s = str(raw)
87          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
88              return s
89          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
99
```

9. assistant (2026-06-21T09:19:37.969Z)

Now I have the full picture of the submit-time gate. Let me trace a `gs://bucket/video.mp4` URI through `process_video()` under default config. First, let me confirm the default config values for `security.allowed_video_root` and the `storage` section.

10. user (2026-06-21T09:19:38.631Z)

```
backend\config\__init__.py
backend\config\presets.py
backend\config\loader.py
backend\config\models.py
```

11. user (2026-06-21T09:19:40.378Z)

```
-rw-r--r-- 1 avidu 197609 2735 Jun 16 01:52 C:/Users/avidu/Projects/badminton-highlight-
indexer/config.json
---default config.json contents---
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-flash-latest",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [
                35,
```

```
            40,
            40
        ],
        "court_green_upper": [
            85,
            255,
            255
        ],
        "court_pixels_min_ratio": 0.05
    }
},
"indexing": {
    "default_segmenter": "gemini",
    "use_gpu": true,
    "motion_threshold": 0.08,
    "min_segment_duration": 3.0,
    "dead_time_merge_gap": 2.0,
    "chunk_duration": 90,
    "chunk_overlap": 10,
    "detector_model": "fasterrcnn_resnet50_fpn",
    "detector_score_threshold": 0.5,
    "yolo_velocity_threshold": 0.15,
    "yolo_frame_skip": 3,
    "yolo_tracker": "bytetrack.yaml",
    "yolo_prefetch_workers": 2,
    "min_action_window_duration": 2.0,
    "log_yolo_candidates": true,
    "store_yolo_candidates": true,
    "ai_handoff_padding": 2.0,
    "ai_max_workers": 1,
    "tracknet": {
        "wsl_distro": "Ubuntu",
        "repo_dir": "~/models/TrackNetV4",
        "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
        "conda_env": "TrackNetV4",
        "weights_path": "",
        "queue_length": 5,
        "stage_in_wsl": true,
        "wsl_stage_dir": "~/clips",
        "timeout_sec": 1800,
        "velocity_threshold": 0.02
    },
    "wasb": {
        "wsl_distro": "Ubuntu",
        "repo_dir": "~/models/WASB-SBDT",
        "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
        "conda_env": "wasb",
        "weights_path": "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
        "sport": "badminton",
        "wsl_stage_dir": "~/clips",
        "timeout_sec": 1800,
        "velocity_threshold": 0.02
    }
},
"output": {
    "default_highlights_name": "highlights.mp4",
    "db_name": "sports_indexer.db"
},
"stitching": {
    "begin_padding": 0.5,
    "end_padding": 0.5,
    "use_cache": false,
    "min_shot_count": 1,
    "watermark": {
        "enabled": true,
```

```
            "text": "Generated by Khelsutra.guru",
            "position": "bottom-right",
            "opacity": 0.85,
            "logo_path": "",
            "font_path": ""
        }
    }
}
```

12. assistant (2026-06-21T09:19:44.159Z)

Key finding: the default config.json has NO `security` section and NO `storage` section. The `default_segmenter` is `"gemini"`. Let me check the config models to confirm the defaults for `security.allowed_video_root` and the `storage` section, and verify what happens when these sections are absent.

13. user (2026-06-21T09:19:45.649Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
```

```
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73          # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74          device: str = "cuda"        # auto | cuda | mps | cpu
75          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
76          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79          keep_frames: bool = False
80          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81          min_free_gb: float = 0.0
82          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87          # Tri-state:
88          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91          stream_video: Optional[bool] = None
92
93
94      class FusionConfig(BaseModel):
95          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
```

```
96
97          Every default reproduces control behaviour: the fusion segmenter persists the
98          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99          true ? the person-cue features, but it does NOT gate the served handoff unless a
100         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102         supplied ? off-court persons (spectators / adjacent courts) are excluded.
103         """
104         # Which trajectory substrate seeds the windows (shuttle stays primary).
105         substrate: Literal["wasb", "tracknet"] = "wasb"
106         # Windowing velocity threshold for the fusion family (the engine reads
107         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109         velocity_threshold: float = 0.02
110         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111         # enabled ? so the default path never imports torch / touches the GPU.
112         cue_flags: dict[str, bool] = Field(default_factory=dict)
113         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115         min_crossings: int = Field(default=0, ge=0)
116         # Served Gate B: when the person cue is on, drop windows with median in-court
players
117         # < this. 0 = OFF.
118         min_players: int = Field(default=0, ge=0)
119         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120         court_polygon: Optional[list[list[float]]] = None
121         # Net line for the person two-sided-motion split (person features only ? the shuttle
122         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123         net_axis: Literal["x", "y"] = "y"
124         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
```

```
151         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155         max_window_duration: float = 6.0
156         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
157         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160         # Only cuts (recall can't drop). OFF by default until measured/promoted.
161         window_hard_cap: bool = False
162         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
```

```
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
        # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
        # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
        # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
        # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
        skip_ai_handoff: bool = False
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
```

```
247         font: str = "Sans"         # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True           # faint backing box behind the text for legibility
251         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
252         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296
297     class StorageConfig(BaseModel):
298         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
299         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
300         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
301         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
```

```
302         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
303         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
304         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
305         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
306         output_root: str = ""
307         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
308         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
309         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
310         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
311         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
312         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
313         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
314         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
315         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
316         input_root: str = ""
317         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
318         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
319         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
320         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
321         # the legacy flat layout is used unchanged.
322         single_folder_per_video: bool = False
323         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
324         workspace_root: str = ""
325         # Cloud namespace prefix under the root so per-video folders sit tidily beside
326         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
327         workspace_prefix: str = "workspaces"
328         local: dict[str, Any] = Field(default_factory=dict)
329         s3: dict[str, Any] = Field(default_factory=dict)
330         gcs: dict[str, Any] = Field(default_factory=dict)
331         gdrive: dict[str, Any] = Field(default_factory=dict)
332
333
334     class DeploymentConfig(BaseModel):
335         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
337
338         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339         behaves exactly as before this section existed. A profile is opt-in (``config.json``
340         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
341         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
342         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
343         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
344
345         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
346
347         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
```

```
348         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
349         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
350         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
351         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
352
353
354     class AppConfig(BaseModel):
355         """Root configuration object.
356
357         All runtime configuration should be accessed via an instance of this class
358         (or the loader) rather than raw dict lookups.
359         """
360
361         sport: Sport = "badminton"
362         ai_provider: str = "gemini"
363         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
364         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
365         ai_model: str = "gemini-flash-latest"
366
367         validation: ValidationConfig = Field(default_factory=ValidationConfig)
368         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
369         output: OutputConfig = Field(default_factory=OutputConfig)
370         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
371         security: SecurityConfig = Field(default_factory=SecurityConfig)
372         logging: LoggingConfig = Field(default_factory=LoggingConfig)
373         storage: StorageConfig = Field(default_factory=StorageConfig)
374         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
375
376         # Allow extra keys during transition so existing config.json files do not break.
377         model_config = {
378             "extra": "allow",
379             "validate_assignment": True,
380         }
381
382         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
383         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
384         # attribute access / the typed config. A nested *section* is returned as a plain
dict
385         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
386         # section model instead breaks it (a model has no .get), which bit every validator
on
387         # real runs. Scalar leaves are returned as-is.
388         def get(self, key: str, default: Any = None) -> Any:
389             _MISSING = object()
390             val: Any = _MISSING
391             if key in type(self).model_fields or hasattr(self, key):
392                 val = getattr(self, key, _MISSING)
393             if val is _MISSING:
394                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
395                 # discovered dynamically from the model fields, so a future top-level
section
396                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
397                 for fname in type(self).model_fields:
398                     section = getattr(self, fname, None)
399                     if isinstance(section, BaseModel) and hasattr(section, key):
400                         val = getattr(section, key)
401                         break
402             if val is _MISSING:
403                 return default
```

```
404             if isinstance(val, BaseModel):
405                 return val.model_dump(mode="json")
406             return val
407
408         def __getitem__(self, key: str) -> Any:
409             val = self.get(key)
410             if val is None and key not in type(self).model_fields:
411                 # Re-raise like a real dict for unknown top-level
412                 raise KeyError(key)
413             return val
414
415
416     # Convenience type alias for callers that want to annotate
417     Config = AppConfig
418
```

14. assistant (2026-06-21T09:19:58.547Z)

Now I have all the defaults confirmed. Let me trace the submit path for `gs://bucket/video.mp4` under default config:

Default config (from config.json + model defaults):
- `security.allowed_video_root` = `None` (no `security` section in config.json ? SecurityConfig default ? `allowed_video_root=None`)
- `storage.input_backend` = `"local"`, `storage.input_root` = `""` (no `storage` section ? StorageConfig defaults)
- `indexing.default_segmenter` = `"gemini"` (from config.json)

Trace of `process_video(req)` with `req.video_path = "gs://bucket/video.mp4"`, `req.segmenter = None`:

1. **Line 269:** `allowed_root = None`
2. **Line 271:** `validate_input_path("gs://bucket/video.mp4", allowed_root=None)` ? only null-byte check ? no null byte ? returns None. PASS.
3. **Line 274:** `resolve_input_ref("gs://bucket/video.mp4", storage_cfg)`:
   - `resolve_input_uri`: `_SCHEME_RE.match("gs://...")` matches (rule 1) ? returns `"gs://bucket/video.mp4"` as-is.
   - `parse_uri("gs://bucket/video.mp4")`: scheme `gs` ? normalized to `gcs`, `gcs` in `_KNOWN` ? returns `StorageRef("gcs", "bucket", "video.mp4", "gs://bucket/video.mp4")`. No ValueError.
4. **Line 283:** `input_ref.backend == "local"` ? `"gcs" == "local"` is **False** ? the `os.path.exists` 404 branch is **SKIPPED**. PASS.
5. **Line 285:** `req.segmenter` is `None` ? falsy ? the unknown-segmenter 400 branch is **SKIPPED**. PASS.
6. **Lines 292-296:** `create_job(...)` ? returns 202 with job_id.

So the submit path does NOT 400/404 a valid `gs://` URI under default config. The claim's submit-gate portion holds. Let me check the edge case where `req.segmenter` is explicitly provided to make sure I'm not missing a branch, and verify `create_job` can't fail for a valid gs path. Let me also double check that there's no other validation issue ? particularly whether `gs://` could be misrouted. Let me verify the `_KNOWN` tuple and check whether there's any case-sensitivity or scheme issue with `gcs://` vs `gs://`.

The lens specifies "ONLY the submit path" and three checks. Let me verify the explicit-segmenter sub-branch (line 285) is the only segmenter gate, and confirm there is no other 4xx between submit and job creation. Let me check `create_job` for completeness since it's the last gate before returning the queryable job.

15. user (2026-06-21T09:20:00.110Z)

```
486:    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487-                   player_pool: Optional[List[str]] = None,
488-                   max_frames: Optional[int] = None,
489-                   policy: Optional[Dict[str, Any]] = None) -> bool:
490-        """Insert a new job in state 'queued'. ``policy`` is an optional JSON ExecutionPolicy
```

```
491-        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492-        return self._execute_write(
493-            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy) "
494-            "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495-            (job_id, video_path, segmenter,
496-             json.dumps(player_pool) if player_pool is not None else None,
497-             max_frames, json.dumps(policy) if policy is not None else None),
498-            f"Failed to create job {job_id}",
499-        )
500-
501-    def mark_job_running(self, job_id: str) -> bool:
502-        return self._execute_write(
503-            "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
504-            "progress='starting' WHERE id = ?", (job_id,),
505-            f"Failed to mark job {job_id} running",
506-        )
507-
508-    def set_job_progress(self, job_id: str, progress: str) -> bool:
509-        return self._execute_write(
510-            "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511-            f"Failed to set progress for job {job_id}",
512-        )
513-
514-    def set_job_video_id(self, job_id: str, video_id: str) -> bool:
515-        return self._execute_write(
516-            "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
517-            f"Failed to set video_id for job {job_id}",
518-        )
519-
520-    def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521-        """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522-        validation) is inside `result` ? job status 'done' means 'the worker finished'."""
523-        return self._execute_write(
524-            "UPDATE jobs SET status='done', progress='finished', result=?, "
525-            "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
526-            (json.dumps(result), job_id),
527-            f"Failed to mark job {job_id} done",
528-        )
529-
530-    def mark_job_failed(self, job_id: str, error: str) -> bool:
531-        """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
```

16. assistant (2026-06-21T09:20:06.069Z)

`create_job` doesn't inspect `video_path` content ? it just inserts. It can only return False on a DB write error (environmental, not a code gate on the URI). The 500 at line 295 is unreachable for a valid gs URI absent a DB fault.

My lens is the SUBMIT-TIME GATE only (the three checks specified). I've traced every branch in `process_video()` (lines 260-300) for a `gs://bucket/video.mp4` URI under default config. Let me do one final confirmation: verify that the default config actually loads without a `security`/`storage` section by checking the loader does not inject a different default, and confirm `getattr(global_config, "security", None)` yields the SecurityConfig (default-factory) not None.

17. user (2026-06-21T09:20:06.866Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
```

```

    from __future__ import annotations

    import json
    import logging
    import os
    from pathlib import Path
    from typing import Any

    from pydantic import BaseModel

    from .models import AppConfig
    from .presets import (
        COMPUTE_TARGET_FOR_PROFILE,
        PROFILE_PRESETS,
        is_known_profile,
        preset_for,
        suggest_profile,
    )

    logger = logging.getLogger(__name__)

    DEFAULT_CONFIG_PATH = Path("config.json")


    def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
        """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
        replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
        overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
        path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
        out = dict(base)
        for key, val in override.items():
            if isinstance(val, dict) and isinstance(out.get(key), dict):
                out[key] = _deep_merge(out[key], val)
            else:
                out[key] = val
        return out


    def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
        """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
        is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
        environment never silently selects a profile (and never silently spends on cloud).

        An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
        profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
        asserting a preset that was never applied. A bad value in config.json is left to
surface as
        a hard, typed-Literal error downstream (config-file correctness)."""
        env_profile = os.environ.get("DEPLOYMENT_PROFILE")
        if env_profile and env_profile.strip():
            ep = env_profile.strip()
            if is_known_profile(ep):
                return ep
            logger.warning(
                "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
                "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
```

```
            )
            # fall through to the file's profile (env typo must not suppress a valid file
choice)
        dep = file_data.get("deployment")
        if isinstance(dep, dict):
            prof = dep.get("profile")
            if isinstance(prof, str) and prof.strip():
                return prof.strip()
        return ""


    def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
        """Dotted paths in `data` that the typed config will not honor.

        Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
        kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
        typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
        user's intent is lost ? collect every such key so load_config can warn.
        """
        unknown: list[str] = []
        fields = model_cls.model_fields
        for key, val in data.items():
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
```

```
`null`) would
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
        # loader's "bad input is non-fatal" contract.
        if not isinstance(file_data, dict):
            logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
                           cfg_path, type(file_data).__name__)
            file_data = {}

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
        # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
        # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
        # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
        defaults = _get_builtin_defaults()
        profile = _resolve_explicit_profile(file_data)

        if profile and not is_known_profile(profile):
            # Only an unrecognised value from config.json reaches here (env typos already
fell back
            # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
            # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
            logger.warning(
                "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
                profile, ", ".join(PROFILE_PRESETS),
            )
            profile = ""

        if profile:
            merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
            # Record the profile actually applied ? the env may have chosen it over
config.json,
            # so re-stamp it onto the merged deployment section to keep the record honest.
            merged.setdefault("deployment", {})
            if isinstance(merged["deployment"], dict):
                merged["deployment"]["profile"] = profile
                ct = merged["deployment"].get("compute_target")
                canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
                if not ct and canonical:
                    # An active profile with no explicit compute_target (e.g. config.json
blanked it
                    # to "") gets its canonical target back ? the record never silently lies
about an
                    # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
                    merged["deployment"]["compute_target"] = canonical
```

```
174                 elif ct and canonical and ct != canonical:
175                     logger.warning(
176                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
177                         "config.json overrides it to %r ? honoring the explicit override.",
178                         profile, canonical, ct,
179                     )
180                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
181                             profile, merged["deployment"].get("compute_target"))
182         else:
183             # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
184             merged = {**defaults, **file_data}
185             suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only
(D15)
186             if suggestion:
187                 logger.debug(
188                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
189                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
190                     "auto-applies ? D15).", suggestion,
191                 )
192
193         return AppConfig.model_validate(merged)
194
195
196     def save_default_config(path: str | Path | None = None) -> Path:
197         """Write a fresh default config.json.
198
199         Used by scripts/doctor.py diagnostics and the --setup CLI path.
200         Overwrites if the file exists.
201         """
202         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
203         cfg = AppConfig()
204         cfg_path.parent.mkdir(parents=True, exist_ok=True)
205         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
206         return cfg_path
207
208
209     # Backwards-compatible alias used by some older call sites during transition.
210     def get_default_config() -> dict[str, Any]:
211         """Return defaults as a plain dict (for transitional use only)."""
212         return _get_builtin_defaults()
213
```

18. assistant (2026-06-21T09:20:19.345Z)

Confirmed. With no `deployment.profile` in config.json (and no `DEPLOYMENT_PROFILE` env), the loader takes the no-profile path (line 184): `merged = {**defaults, **file_data}`. Since `file_data` (config.json) has no `security` and no `storage` keys, the defaults supply them, and `AppConfig.model_validate(merged)` instantiates `SecurityConfig` (allowed_video_root=None) and `StorageConfig` (input_backend="local", input_root=""). So `getattr(global_config, "security", None)` is a real SecurityConfig and `getattr(global_config, "storage", None)` is a real StorageConfig ? both yielding the byte-identical defaults I traced.

I have fully verified the submit-time gate. Let me record my final assessment.

Summary of the trace (default single-user config, `req.video_path="gs://bucket/video.mp4"`, `req.segmenter=None`):

Submit gate	Location	Outcome for gs://
`validate_input_path`	`backend/utils/paths.py:53-63`, called at `backend/main.py:271`	`allowed_root=None` ? only null-byte check ? PASS (no 400)

`resolve_input_ref` ValueError?400	`backend/storage/__init__.py:38-48` + `ref.py:71-98,48-68`, called at `backend/main.py:274-276`	`gs://` matches scheme rule 1 (returned as-is), `parse_uri` normalizes `gs`?`gcs` (in `_KNOWN`) ? returns `StorageRef("gcs",...)`, NO ValueError ? no 400
`input_ref.backend=="local"` existence 404	`backend/main.py:283-284`	backend is `"gcs"`, not `"local"` ? branch SKIPPED ? no 404
explicit-segmenter 400	`backend/main.py:285-290`	`req.segmenter` is None ? falsy ? branch SKIPPED ? no 400
`create_job` ? 500	`backend/main.py:293-295`	inserts unconditionally; only fails on a DB write fault (environmental), not on URI content

Every submit-time gate passes; the endpoint returns 202 + a queryable `job_id`. No code branch in the submit path 400s/404s a valid `gs://` URI under default config. I could not find a counterexample. The only failure modes are environmental (missing `google-cloud-storage` / ADC at fetch time), which the lens explicitly excludes, and those happen on the WORKER, not the submit path anyway.

19. user (2026-06-21T09:20:31.861Z)

Structured output provided successfully

20. assistant (2026-06-21T09:20:39.424Z)

I traced the submit-time gate exhaustively and could not refute the claim. Under default single-user config a valid `gs://` URI passes all four submit gates and yields a queryable 202 + job_id.

Key files:
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:260-300` ? the submit path; all four gates skip/pass for a gs:// URI
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/paths.py:53-63` ? `validate_input_path` (null-byte only when `allowed_root=None`)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/storage/ref.py:48-68,71-98` ? `parse_uri` normalizes `gs`?`gcs` (no ValueError) and `resolve_input_uri` returns the explicit scheme as-is
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/loader.py:184` + `config/models.py:280-331` ? default config (no `security`/`storage` section) yields `allowed_video_root=None`, `input_backend="local"`

refuted=false; no concrete code-gate counterexample. The only failure paths are environmental (missing google-cloud-storage / ADC at worker fetch), which the lens excludes and which occur off the submit path anyway.